

Logic Design and Switching Theory

Lecture – 4

Subtitle: SOP and POS, Gate level minimization of Boolean Functions using K Map

by
Ms. Hera Noor

What's new!

- **Minterm & Maxterm**
- **SOP & POS**
- **Consensus Theorem**
- **Duality Theorem**
- **Structure of K-Map**
- **Rules of Grouping**
- **Don't Care Condition**
- **5 variable K-map**
- **Variable Entrant Map**

Previous

1. Identity Laws:

- $A + 0 = A$
- $A \cdot 1 = A$

2. Null Laws:

- $A + 1 = 1$
- $A \cdot 0 = 0$

3. Idempotent Laws:

- $A + A = A$
- $A \cdot A = A$

4. Complement Laws:

- $A + A' = 1$
- $A \cdot A' = 0$

5. Double Negation Law:

- $(A')' = A$

6. Distributive Laws:

- $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
- $A + (B \cdot C) = (A + B) \cdot (A + C)$

7. Absorption Laws:

- $A + (A \cdot B) = A$
- $A \cdot (A + B) = A$

8. De Morgan's Theorems:

- $(A \cdot B)' = A' + B'$
- $(A + B)' = A' \cdot B'$

9. Consensus Theorem:

- $(A + B)(A + B') = A + B$
- $(A \cdot B) + (A \cdot B') = A$

10. Universal Bound Laws

- $A + A'B = A + B$
- $A \cdot (A' + B) = A \cdot B$

11. Redundancy (Covering) Theorem

- $A + AB = A$
- $A(A + B) = A$

Minterms and Maxterms

<i>Variables</i>			<i>Minterms</i>	<i>Maxterms</i>
<i>A</i>	<i>B</i>	<i>C</i>		
0	0	0	$\bar{A}\bar{B}\bar{C} = m_0$	$A + B + C = M_0$
0	0	1	$\bar{A}\bar{B}C = m_1$	$A + B + \bar{C} = M_1$
0	1	0	$\bar{A}B\bar{C} = m_2$	$A + \bar{B} + C = M_2$
0	1	1	$\bar{A}BC = m_3$	$A + \bar{B} + \bar{C} = M_3$
1	0	0	$A\bar{B}\bar{C} = m_4$	$\bar{A} + B + C = M_4$
1	0	1	$A\bar{B}C = m_5$	$\bar{A} + B + \bar{C} = M_5$
1	1	0	$AB\bar{C} = m_6$	$A + B + \bar{C} = M_6$
1	1	1	$ABC = m_7$	$\bar{A} + \bar{B} + \bar{C} = M_7$

Representation of Minterms and Maxterms

$$\begin{aligned} 1. \quad Y &= \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}C + ABC \\ &= m_0 + m_2 + m_5 + m_7 \\ &= \sum m(0, 2, 5, 7) \end{aligned}$$

$$\begin{aligned} 2. \quad Y &= (A + B + C)(\overline{A} + B + C)(\overline{A} + \overline{B} + \overline{C}) \\ &= M_1 + M_4 + M_7 \\ &= \prod M(1, 4, 7) \end{aligned}$$

where $\sum$ and $\prod$ denote the sum of product and product of sum respectively.

SOP & POS

In SOP or Sum of Product, form variables are ANDed(i.e multiplied) together and used to interpret outputs with logic '1' combination. In SOP form each product term is identified as a minterm.

Example: – $ABC' + A'BC + ABC$
110 011 111

In POS or Product of Sum, form variables are ORed(i.e added) together and applied to represent outputs with logic '0' combination. In POS form every single term is known as the maxterm.

Example: – $(A+B'+C)(A'+B+C')(A+B+C)$
010 101 000

Consensus Theorem

The Consensus Theorem eliminates redundant terms in a Boolean expression.

$$\mathbf{AB + A'C + BC = AB + A'C}$$

The consensus or resolvent of the terms AB and $A'C$ is BC . It is the conjunction of all the unique literals of the terms, excluding the literal that appears unnegated in one term and negated in the other.

The conjunctive dual of the theorem is:

$$\mathbf{(A+B).(A'+C).(B+C) = (A+B).(A'+C)}$$

$$F = \underbrace{(A + B).(A' + C)}_{\text{Complemented variable}}.(B + C) \text{ --- Redundancy term}$$

$$F = \underbrace{AB}_{\text{Redundancy term}} + \underbrace{BC' + AC}_{\text{Complemented Variable}}$$

Proof

$$Y = AB + A'C + BC$$

$$Y = AB + A'C + BC.1$$

$$Y = AB + A'C + BC.(A + A')$$

$$Y = AB + A'C + ABC + A'BC$$

$$Y = AB(1 + C) + A'C(1 + B)$$

$$Y = AB + A'C$$

Duality Theorem

The Duality Theorem states that every Boolean algebra equation remains valid if:

- The AND and OR operators are interchanged.
- The constants 0 and 1 are swapped.

	Function	Dual
Commutation	$A + B = B + A$	$AB + BA$
Association	$A + (B + C) = (A + B) + C$	$A(BC) = (AB)C$
Distribution	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption	$A + AB = A$	$A(A + B) = A$
De Morgan	$\overline{A+B} = \bar{A} \cdot \bar{B}$	$\overline{AB} = \bar{A} + \bar{B}$
Consensus	$AC + B\bar{C} =$ $AB + AC + B\bar{C}$	$(A + C)(B + \bar{C}) =$ $(A + B)(A + C)(B + \bar{C})$

Comparison

Aspect	Consensus Theorem	Duality Theorem
Purpose	Removes redundant terms	Generates dual expressions
Simplification	Reduces logic complexity	Verifies and derives equations
Application	• Digital circuit optimization • Simplifying combinational circuits. • Reducing gate count in designs.	• Logical equivalence validation • Ensuring logical consistency. • Creating complementary logic circuits.

Examples

- Design a circuit is to be 1 if and only if the inputs represent a number that is either 0 or a power of 2.
- Design a circuit whose output is to be 1 if and only if the 3 bit inputs represent a number that is completely divisible by 2 or 5.

Solve these!

Complete the truth tables for the given Boolean expressions:

$$\text{Output} = \overline{A} + \overline{B} + C$$

A	B	C	Output
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Solve these!

Complete the truth tables for the given Boolean expressions:

$$\text{Output} = A(B + AC + \overline{A})$$

A	B	C	Output
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Structure of a K-map

- K-maps are grids where each cell represents a **minterm** (binary combination of input variables).
- The number of cells equals 2^n , where n is the number of input variables:
 - ✓ 2 variables $\rightarrow$ 4 cells.
 - ✓ 3 variables $\rightarrow$ 8 cells.
 - ✓ 4 variables $\rightarrow$ 16 cells.

Example

Two variable maps

A \ B	0	1
0	0	1
1	1	0

$$F = A\bar{B} + \bar{A}B$$

A \ B	0	1
0	0	1
1	1	1

$$F = AB + \bar{A}B + A\bar{B}$$
$$F = A + B$$

Three variable maps

A \ BC	00	01	11	10
0	0	1	0	1
1	1	1	1	1

$$F = A + \bar{B}\bar{C} + B\bar{C}$$

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + A\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC$$

Four variable maps

		CD			
		00	01	11	10
AB	00	0	0	0	1
	01	1	1	0	1
	11	1	1	1	1
	10	1	0	1	1

$$F = A'BC' + A'CD' + ABC \\ + AB'C'D' + ABC' + AB'C$$

$$F = BC' + AC + CD' + AD'$$

Need to make sure all 1's are covered

Try to minimize total product terms

Design could be implemented using NANDs and NORs

Rules for Grouping

- Groups must contain 1,2,4,8...cells (powers of 2).
- Groups can wrap around the edges of the K-Map.
- Overlapping groups are allowed and sometimes necessary.
- Each group should be as large as possible while including only 1s.

Prime Implicant

Any single 1 or group of 1s in the Karnaugh map of a function F is an implicant of F .

A product term is called a prime implicant of F if it cannot be combined with another term to eliminate a variable.

AB \ CD	CD			
	00	01	11	10
00	1		1	1
01			1	1
11	1			
10	1	1		

- (a) $A'B'C$
- (b) BD
- (c) $A'B'C'D'$
- (d) $A'C$
- (e) $A'B'D'$

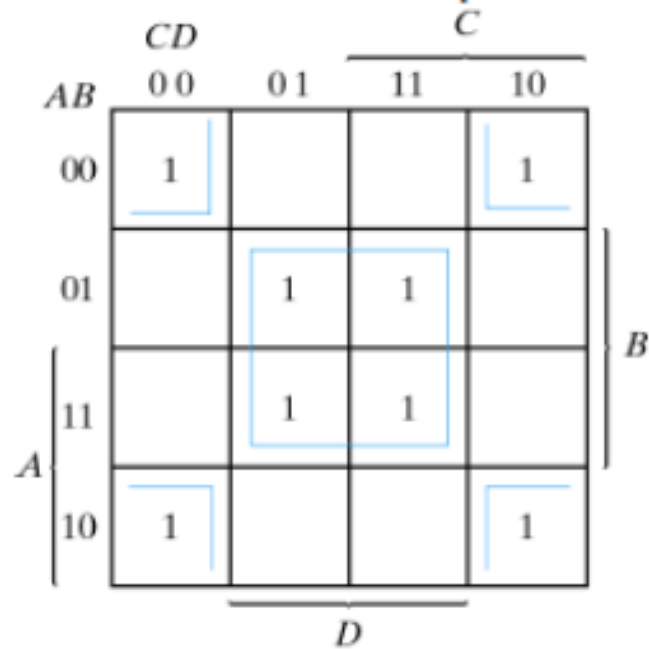
Implicants:
(a),(c),(d),(e)

Prime Implicants:
(d),(e)

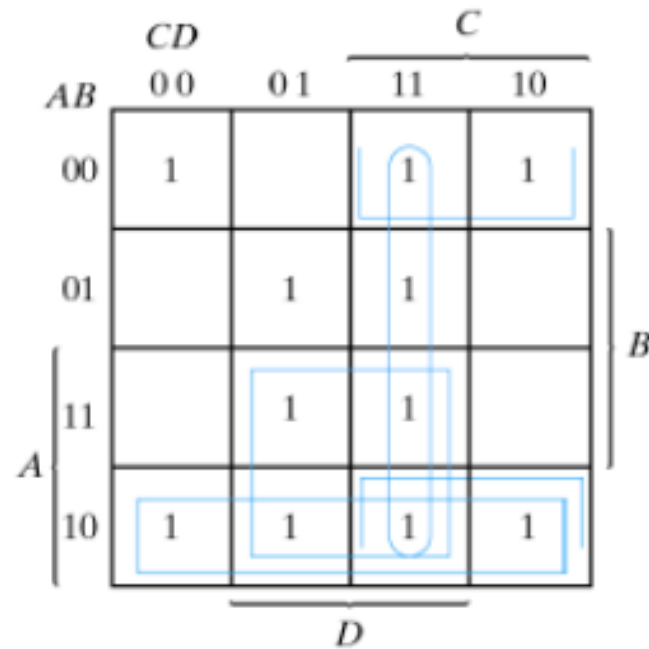
Essential Prime Implicant

A product term is an essential prime implicant if there is a minterm that is only covered by that prime implicant

The minimal sum-of-products form of F must include all the essential prime implicants of F



(a) Essential prime implicants
 BD and $B'D'$



(b) Prime implicants CD , $B'C$
 AD , and AB'

Prime Implicant - A prime implicant is the largest possible group of adjacent 1s in a K-map that can be combined into a single term in the simplified Boolean expression.

- It is a minimal product term (ANDed variables) that cannot be combined further.
- Covers one or more minterms.

Essential PI - An essential prime implicant is a prime implicant that covers at least one minterm that no other prime implicant covers.

- Essential prime implicants are critical to include in the simplified Boolean expression.
- They ensure all minterms are covered.

Rules for Grouping

Redundant PI - A redundant prime implicant is a prime implicant that is not necessary for covering any minterms, as all its minterms are already covered by other essential prime implicants.

- It can be omitted without affecting the final Boolean expression.

Relationship Between the Terms

Prime Implicants: All possible groups of adjacent 1s.

Essential Prime Implicants: Prime implicants that cover unique minterms.

Redundant Prime Implicants: Prime implicants that are not essential and whose minterms are covered by others.

Don't Care Condition

In some practical logic designs, there are combinations of input variable that we know will never occur.

These combinations are marked as 'd' or 'x'.

In k map, these can be mapped treating as '1' to form larger groups or can be disregarded treating as '0'.

Since it doesn't matter whether to be treated as '1' or '0', these are called don't care.

Don't Care Condition

An 8421 binary coded decimal is a 4 bit coding system. Codes from 0000-1001 are used to represent numbers from 0 to 9. Remaining 6 unused combinations 1010-1111 are don't care conditions.

Decimal	Binay (BCD)			
	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Don't Care Condition

In some cases, outputs are undefined

We “don't care” if the circuit produces a '0' or a '1'

This knowledge can be used to simplify functions

		A			
		00	01	11	10
CD	00	0	0	X	0
	01	1	1	X	1
	11	1	1	0	0
	10	0	X	0	0

Diagram labels: A horizontal line above the columns is labeled 'A'. A vertical line to the left of the rows is labeled 'C'. A horizontal line below the columns is labeled 'B'. A vertical line to the right of the rows is labeled 'D'. The labels 'AB' and 'CD' are at the top left of the grid.

- Treat X's like either 1's or 0's
- Very useful
- OK to leave some X's uncovered

Example

$$F(A,B,C,D) = \sum m(0, 2, 3, 5, 6, 7, 8, 10, 11, 14, 15)$$



		CD		C	
		00	01	11	10
AB	00	1	0	1	1
	01	0	1	1	1
	11	0	0	1	1
	10	1	0	1	1

Diagram illustrating the Karnaugh map for the function $F(A,B,C,D)$. The map is a 4x4 grid with rows labeled AB (00, 01, 11, 10) and columns labeled CD (00, 01, 11, 10). The function is 1 for minterms 0, 2, 3, 5, 6, 7, 8, 10, 11, 14, 15. The map shows four groups of 1s circled in red: a group of 4 (C), a group of 4 (A'BD), and two groups of 2 (B'D').

$$F = C + A'BD + B'D'$$

Example

Design a logic circuit that produces 1 when the input BCD digit is an odd number.

Five people judge a certain competition. the vote of each is indicated by '1' (pass) or '0' (fail) on a signal line. The five signal lines form an input to the logic circuit. The rule of competition allow only one dissenting vote. The logic circuit is to have 2 outputs, x and y. If the vote is 4-1 or 5-0 to pass, $xy = 11$. If the vote is 4-1 or 5-0 to fail, $xy = 00$. If the vote 3-2 or 2-3, $xy=10$. Design the logic circuit for this problem?

5 variable k-map

$$F(A,B,C,D,E) = \sum(0,2,4,6,9,11,13,15,17,21,25,27,29,31) = BE + AD'E + A'B'E'$$

A \ B		CDE							
		000	001	011	010	110	111	101	100
0	00								
	01								
	11								
	10								

K-Map for LT, EQ and GT

Inputs and Outputs

1.Inputs: Two binary numbers A and B .

Assume $A = A_1A_0$ and $B = B_1B_0$, where A_1, A_0 and B_1, B_0 are bits of A and B , respectively.

Total combinations: $2^4 = 16$ (since there are 4 bits in total).

2.Outputs:

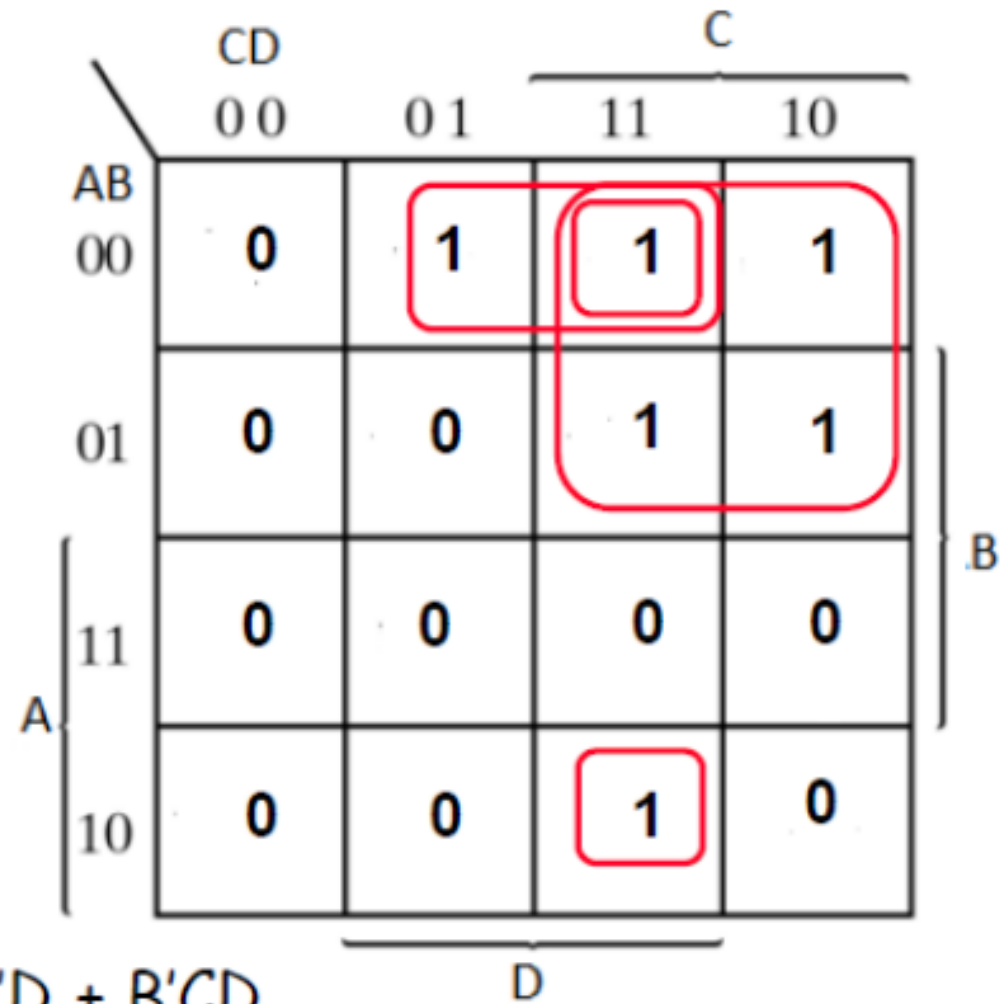
$$LT = 1: A < B$$

$$EQ = 1: A = B$$

$$GT = 1: A > B$$

A ₁ A ₀	B ₁ B ₀	LT	EQ	GT
00	00	0	1	0
00	01	1	0	0
00	10	1	0	0
00	11	1	0	0
01	00	0	0	1
01	01	0	1	0
01	10	1	0	0
01	11	1	0	0
10	00	0	0	1
10	01	0	0	1
10	10	0	1	0
10	11	1	0	0
11	00	0	0	1
11	01	0	0	1
11	10	0	0	1
11	11	0	1	0

K-Map for LT



K-Map for EQ

		CD		C	
		0 0	0 1	1 1	1 0
AB	0 0	1	0	0	0
	0 1	0	1	0	0
	1 1	0	0	1	0
	1 0	0	0	0	1

A bracket on the left side of the table groups the rows for AB = 11 and 10, labeled 'A'. A bracket on the right side groups the columns for C = 11 and 10, labeled 'B'.

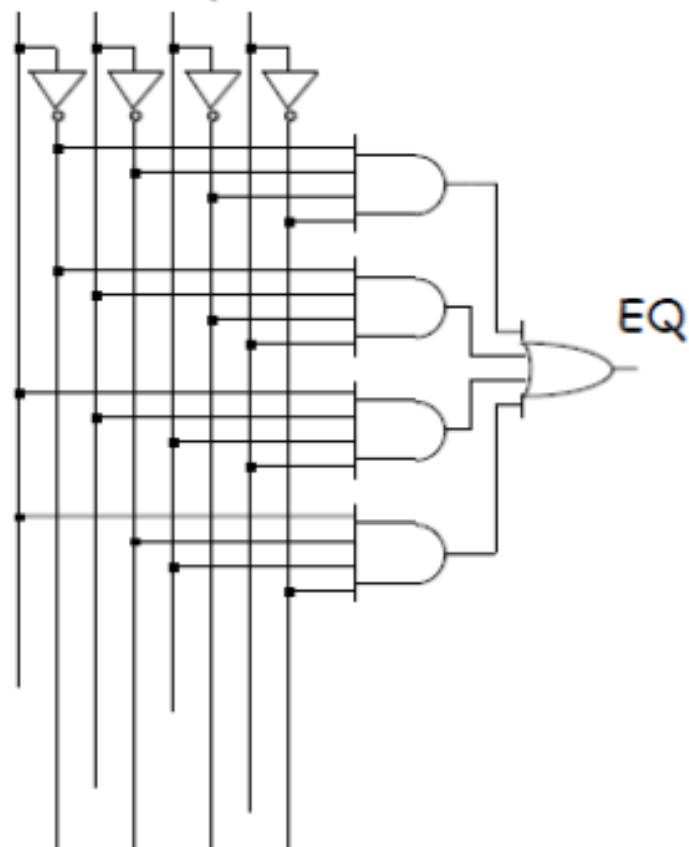
$$F = A'B'C'D' + A'BC'D + ABCD + AB'CD'$$

K-Map for GT

		CD		C	
		0 0	0 1	1 1	1 0
AB	0 0	0	0	0	0
	0 1	1	0	0	0
	1 1	1	1	0	1
	1 0	1	1	0	0

$F = AC' + BC'D' + ABD'$

Physical Implementation



		CD		C	
		0 0	0 1	1 1	1 0
A	AB	1	0	0	0
	00	0	1	0	0
	01	0	0	1	0
	11	0	0	0	1
		D			
		B			

$$F = A'B'C'D' + A'BC'D + ABCD + AB'CD'$$

Variable Entered Map

Variable entered mapping (VEM) is used to reduce the size of K-maps with many variables.

In VEM, one variable is chosen as the map entered variable and included in the K-map along with zeros, ones, and don't cares.

This allows representing functions with more variables using a smaller K-map.

How to minimize!

- Write all the variables (original and complimented forms are treated as two different variables) in the map as 0, leave 0's, minterms and don't cares as it is and obtain the SOP expression.
- Select one variable and make all occurrences of that variable as 1, write minterms (1's) as don't cares, leave 0's and don't cares as it is. Now, obtain the SOP expression.
- Multiply the obtained SOP expression with the concerned variable.
- Repeat step 2 for all the variables in the k-map.
- SOP of VEM is obtained by taking OR of all the obtained SOP expressions..

Summary!

Key Features:

- Number of Cells: 2^n (where n is the number of variables).
Example: 2 variables = 4 cells, 3 variables = 8 cells, etc.
- Gray Code Ordering: Adjacent cells differ by only one bit.
- Inputs and Outputs: Rows and columns represent variable combinations, while cells show output (1, 0, or don't care).

Summary!

Steps to Simplify

- **Using K-Maps**
 - ✓ Fill the K-Map: Place 1s, 0s, and don't-cares (d) from the truth table into the grid.
- **Group the 1s:**
 - ✓ Create groups of 1, 2, 4, 8, etc. cells.
 - ✓ Groups must be rectangular and powers of 2.
- **Write the Simplified Expression:**
 - ✓ Each group corresponds to a product term (AND operation).
 - ✓ Combine terms with OR to get the simplified result.

Solve these!

Determine which rule (or rules) are being used in the following Boolean reductions

$$\overline{D}F + \overline{D}FC = \overline{D}F$$

$$1 + G = 1$$

$$B + AB = B$$

$$\overline{F}E + \overline{F}E = \overline{F}E$$

$$XYZ + \overline{X}\overline{Y}\overline{Z} = 1$$

$$GQ + Q = Q$$

$$\overline{H} \overline{H} = \overline{H}$$

Solve these!

Determine which rule (or rules) are being used in the following Boolean reductions

$$\overline{RS} \overline{SR} = \overline{RS}$$

$$ABC\overline{D} + D = D + ABC$$

$$AC\overline{B} + CAD\overline{B} = A\overline{B}C$$

$$A + T + \overline{W} + \overline{A} + X = 1$$

$$X\overline{Y}\overline{Z} + \overline{X} = \overline{X} + \overline{Y}\overline{Z}$$

$$\overline{GFH} \overline{HGF} = \overline{FHG}$$

$$C\overline{A}\overline{B} + AB = AB + C$$

Thank you!